Instruction Following with Dolly-15k

Hala alotaibi

Date: Thursday ,12 march

1. Introduction

Large language models are widely used for instruction-following tasks, but their performance depends heavily on prompt design, data structure, and training strategy. In this project, I explored instruction-following performance using the Dolly-15k dataset by comparing three approaches: zero-shot prompting, few-shot prompting, and fine-tuning a small open-source model. The main objective was to evaluate how well different strategies help an LLM follow instructions under limited data and compute resources.

2. Dataset Description

I used the Dolly-15k dataset from Hugging Face, which contains around 15,000 human-written instruction-response pairs designed for instruction tuning. Each example includes an instruction, an optional context, a response, and a category label. The dataset covers multiple task types such as open question answering, closed question answering, classification, summarization, information extraction, brainstorming, and creative writing.

During exploration, I observed that instructions vary significantly in length and complexity, and many examples do not include context. Some tasks are harder for an LLM due to long instructions, missing context, or tasks requiring reasoning rather than simple recall.

3. Exploratory Data Analysis (EDA)

To better understand the dataset structure, I analyzed categories, text lengths, and context availability. I found that the dataset is diverse and slightly imbalanced across categories. Instructions and responses vary widely in length, and many examples contain no context. Long instructions and long expected responses often make tasks more challenging for models.

These findings influenced my design choices, especially for prompt formatting and evaluation, since some instructions require clearer structure or additional guidance.

4. Prompt Design Strategy

I designed prompts to ensure clarity and consistency for the model. For zero-shot prompting, I used a structured format that included the instruction and context when available. If no context existed, only the instruction was included. This ensured consistency while avoiding unnecessary noise.

I also handled edge cases such as long context by keeping prompts concise and ensuring they stayed within token limits. Empty context was simply omitted from the prompt rather than replaced with filler text.

5. Zero-Shot vs Few-Shot Approach

I compared zero-shot and few-shot prompting to analyze when additional examples improve performance. Few-shot prompts included one to three instruction-response examples selected from the dataset. I selected examples from stratified samples to maintain diversity across categories.

However, results showed that few-shot prompting did not consistently improve performance. In some cases, added examples introduced noise or mismatched patterns that confused the model rather than guiding it.

6. Pipeline Implementation

I built a pipeline that feeds instructions (and context) into an LLM and collects responses automatically. The pipeline supports both zero-shot and few-shot modes. I tested the approaches on a held-out evaluation subset to ensure fair comparison across methods.

7. Evaluation Methodology

I evaluated performance using two approaches: an automatic metric and LLM-as-judge evaluation.

First, I used BLEU to measure similarity between model outputs and reference responses from the dataset. While BLEU is fast and consistent, it cannot fully capture answer quality or reasoning.

Second, I used an LLM-as-judge approach that evaluates responses based on instruction-following quality. This method captures deeper understanding but requires more compute and may introduce subjectivity. Using both methods provided more reliable results.

8. Fine-Tuning Setup

In addition to prompting approaches, I fine-tuned a small open-source model (TinyLlama-1.1B) using LoRA on a subset of 400 samples from Dolly-15k. I formatted the data in a chat-style template combining instruction, optional context, and response.

To reduce overfitting, I limited training epochs and used a small learning rate. Despite this, fine-tuning performed worse than prompting due to limited data and compute resources.

9. Results and Discussion

The results showed that zero-shot achieved the best overall performance in this project. Few-shot performed slightly worse, and fine-tuning performed the weakest. These findings suggest that prompt-based methods can outperform fine-tuning when resources are limited.

Fine-tuning likely underperformed due to the small training subset and limited training time. Additionally, the diversity of tasks in the dataset made it harder for a small model to generalize effectively.

10. Category Diversity and Imbalance

Instruction diversity and category imbalance affected my pipeline design. To address this, I used stratified sampling by category for both prompting experiments and evaluation. This ensured fair comparison and avoided bias toward dominant categories.

11. Cloud vs Local LLM Trade-offs

Cloud LLMs offer strong performance, scalability, and ease of use but come with higher cost and lower reproducibility. Local LLMs provide more control and reproducibility but require more setup and often deliver lower performance with limited hardware. For this project, cloud-based models were more practical for evaluation.

12. Tracking Experiments

I tracked performance across approaches using consistent evaluation subsets and metrics. This allowed me to compare prompt designs and model choices fairly and justify which approach worked best.

13. Conclusion

In conclusion, zero-shot prompting performed best in this setup, showing that careful prompt design can outperform fine-tuning under limited resources. Few-shot provided mixed results, and fine-tuning requires more data and compute to be effective. Combining automatic metrics with LLM-based evaluation provided a more complete understanding of model performance.